Smart Queue Management System

Advanced Data Structures & Algorithms Report

CSAI 201 - Data Structures and Algorithms
Zewail City of Science and Technology

Team Members

Malak Osama - 202402421
Rowida Sayed - 202402291

Introduction

Project purpose, system scope, and implemented features

The Smart Queue Management System is a C++ console-based project that applies core data structures to manage service queues fairly and efficiently. The system assigns a calculated priority to each person using urgency, waiting time, and service type, then serves the highest-priority person first through a max-heap priority queue. It also supports dynamic priority updates, fairness boosting, multiple service queues, admin-controlled settings, simulation mode, file loading, and reporting.

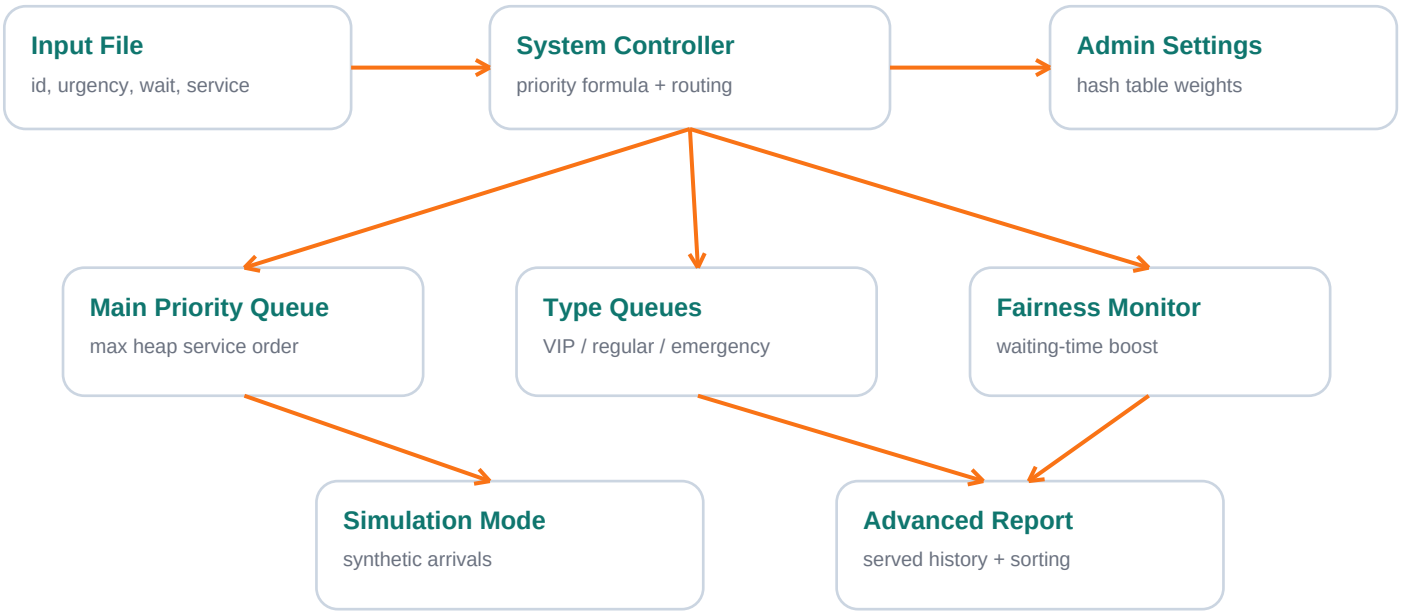
Max Heap	C++	100	3
MAIN DATA STRUCTURE	CORE LANGUAGE	QUEUE CAPACITY	PRIORITY FACTORS

Main Contributions of the System

- Priority-based service ordering using an array-backed max heap.
- Dynamic priority recalculation when waiting time changes.
- Fairness monitor that increases priority for users waiting longer than the allowed threshold.
- Separate VIP, regular, and emergency queues with queue merging support.
- Admin console implemented using a hash table with linear probing.
- Simulation mode and advanced reporting for served users.

1. System Architecture

The implementation is organized around three main classes: Person, PriorityQueue, and SystemController. Person stores the user attributes, PriorityQueue controls max-heap ordering, and SystemController connects all features together, including priority calculation, fairness monitoring, settings, queues, simulation, loading from files, and reports.



Architecture Notes

Component	Responsibility	Main Data Structure
Person	Stores ID, urgency, waiting time, service time, service type, and computed priority.	Object / record
PriorityQueue	Maintains service order so the maximum priority person is extracted first.	Array-backed max heap
SystemController	Controls the full workflow, settings, queues, fairness, simulation, and reports.	Priority queues + hash table + arrays
Admin Settings	Stores weights and thresholds used by the priority engine.	Hash table with linear probing

2. Data Structure Selection

The project combines more than one data structure because the problem has different requirements: fast retrieval of the highest-priority person, fast settings access, separate queue management, and ordered reporting after service completion.

Feature	Selected Structure	Reason	Complexity
Priority engine	Priority Queue / Max Heap	Always serves the highest priority person first.	Insert O(log n), Extract O(log n), Max O(1)
Dynamic updates	Max Heap reordering	Repositions a person when waiting time or priority changes.	Update O(n + log n) due to linear search by ID
Admin console	Hash Table	Stores weights and thresholds with fast lookup.	Average O(1)
Multiple queues	Several priority queues	Separates VIP, regular, emergency, and main service queues.	Merge O(n log n)
Fairness monitor	Priority queue + threshold variables	Boosts waiting users and updates their heap position.	O(n + log n)
Reporting	Array + sorting	Stores served users and sorts them by waiting time.	O(n log n)

Design Decision

A priority queue is the best match for this problem because queue order is not based on arrival time only. Emergency level, waiting time, and service type all affect who should be served next. A max heap keeps this decision efficient and structured.

3. Modular Priority Engine

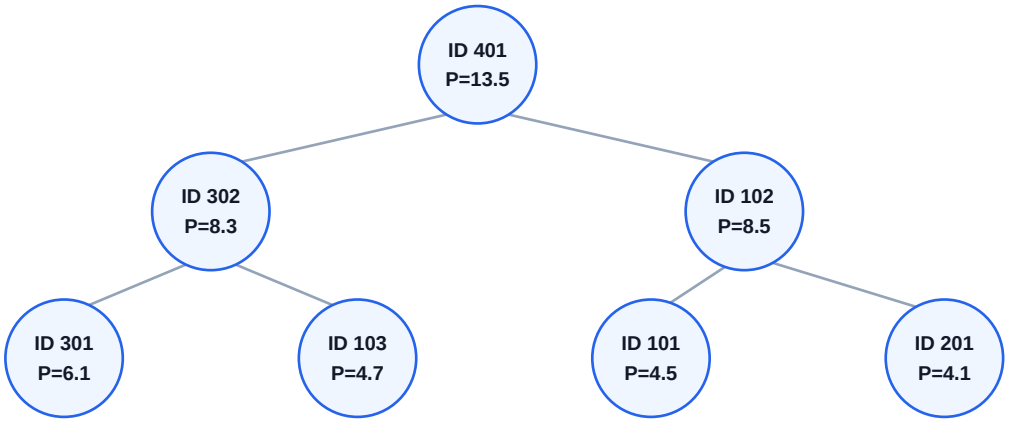
Each person receives a priority score computed from three weighted factors. The weights are not hard-coded inside the formula; they are stored inside the admin hash table so the system can update them at runtime.

Factor	Meaning	Default Weight / Score
Urgency	How critical the case is.	urgency_weight = 0.5
Waiting Time	How long the person has been waiting.	waiting_weight = 0.3
Service Type	Emergency, VIP, or regular service score.	service_weight = 0.2
Service Scores	Emergency = 10, VIP = 8, Regular = 5	stored in hash table

Priority Formula

```
priority = (urgency x urgency_weight) + (waiting_time x waiting_weight) + (service_score x service_weight)
```

Example: ID 101 has urgency = 5, waiting time = 0, and service type = emergency. With weights 0.5, 0.3, and 0.2, the priority is: (5 x 0.5) + (0 x 0.3) + (10 x 0.2) = 4.5.



4. Dynamic Priority Updates and Fairness Monitor

A real queue changes over time. A regular person who has waited too long may become more urgent than a newly arrived VIP. The system handles this by recalculating priority and then restoring heap order using heapifyUp and heapifyDown.

Case	Before	After	Result
Dynamic update	ID 102, urgency 3, wait 10, regular -> priority 5.5	Waiting becomes 20 -> priority 8.5	The heap position is updated.
Fairness boost	ID 401 wait = 30, max_wait = 25, multiplier = 0.5	Extra wait = 5, boost = 2.5	New priority = current priority + 2.5.

Fairness Formula

```
if waiting_time > max_wait_time:
extra_wait = waiting_time - max_wait_time
fairness_boost = extra_wait x boost_multiplier
new_priority = current_priority + fairness_boost
```

This prevents starvation, where low-priority users wait forever because high-priority users keep arriving.

5. Multiple Queues and Admin Console

The system supports four queues: the main queue, VIP queue, regular queue, and emergency queue. Users are inserted into the main queue for service ordering and also routed into a type-specific queue. This allows the system to simulate different service counters and merge queues when needed.

Queue	Purpose
Main queue	Global service order according to calculated priority.
VIP queue	Stores VIP users separately for VIP counter handling.
Regular queue	Stores regular users separately and can be merged into VIP counter if needed.
Emergency queue	Stores emergency users separately.

Hash Table Settings

The admin console stores system settings using arrays for keys, values, and used flags. The hash function sums the characters of the key and applies modulo SETTINGS_SIZE. Linear probing resolves collisions.

Setting Key	Default Value	Use
urgency_weight	0.5	Controls urgency contribution.
waiting_weight	0.3	Controls waiting time contribution.
service_weight	0.2	Controls service type contribution.
max_wait_time	25	Fairness threshold.
boost_multiplier	0.5	Fairness boost multiplier.

6. File Loading, Simulation, and Reporting

The program loads test cases from a text file selected through a Windows file dialog. Each valid line contains: ID, urgency, waiting time, service time, and service type. The first line may optionally contain the number of records and is skipped if detected.

Sample Input File

```
7
101 5 0 6 emergency
102 3 10 8 regular
103 4 5 7 vip
401 2 30 10 regular
201 4 3 6 vip
301 3 12 9 regular
302 2 18 11 regular
```

Module	Implementation Summary
Simulation mode	Generates new regular arrivals over time, computes their priority, inserts them into the main queue, and serves after each arrival.
Advanced reporting	Stores served people in an array, sorts them by waiting time descending, and prints ID, type, waiting time, and service time.
File validation	Skips empty lines, checks wrong line format, and prints an error if the file cannot be opened.

7. Implementation Highlights

The code is modular and separated into header and source files. This improves readability and makes each class responsible for a specific part of the system.

File	Purpose
Person.h / Person.cpp	Defines the Person class and constructors.
PriorityQueue.h / PriorityQueue.cpp	Implements max heap operations: insert, extractMax, updatePriority, heapifyUp, heapifyDown.
SystemController.h / SystemController.cpp	Coordinates priority calculation, settings, fairness, queues, simulation, reports, and file loading.
main.cpp	Runs the complete demonstration flow and connects the project modules.

Complexity Summary

Operation	Complexity	Explanation
Insert person	O(log n)	Heapify up after insertion.
Serve next	O(log n)	Extract root then heapify down.
Access highest priority	O(1)	Highest priority person is stored at heap root.
Update priority	O(n + log n)	Search by ID then reorder heap.
Merge queues	O(n log n)	Extract from one queue and insert into another.
Sort report	O(n log n)	Sort served users by waiting time.
Get / set setting	Average O(1)	Hash table lookup with linear probing.

8. Evaluation and Conclusion

The final system demonstrates how data structures can solve a practical queue management problem. Priority queues provide efficient service ordering, hash tables provide fast settings management, and arrays support reporting and history storage. The design also handles practical concerns such as changing priorities, fairness for long-waiting users, different queue types, simulation, and file-based testing.

Requirement	Status	Evidence
Serve highest priority first	Implemented	Max heap extracts the highest-priority person.
Dynamic priority update	Implemented	updatePriority recalculates and reorders the heap.
Fairness monitor	Implemented	Waiting-time boost increases priority after max_wait_time.
Multiple queues	Implemented	VIP, regular, emergency, and main queues are supported.
Admin settings	Implemented	Hash table stores weights and thresholds.
Simulation and reporting	Implemented	Generated arrivals and sorted served-user report.

Final Statement

Overall, the Smart Queue Management System is a strong data structures project because it connects priority queues, hash tables, arrays, sorting, file handling, and object-oriented design into one working application. The system is practical, explainable, and suitable for demonstrating Data Structures and Algorithms concepts in a project discussion.